

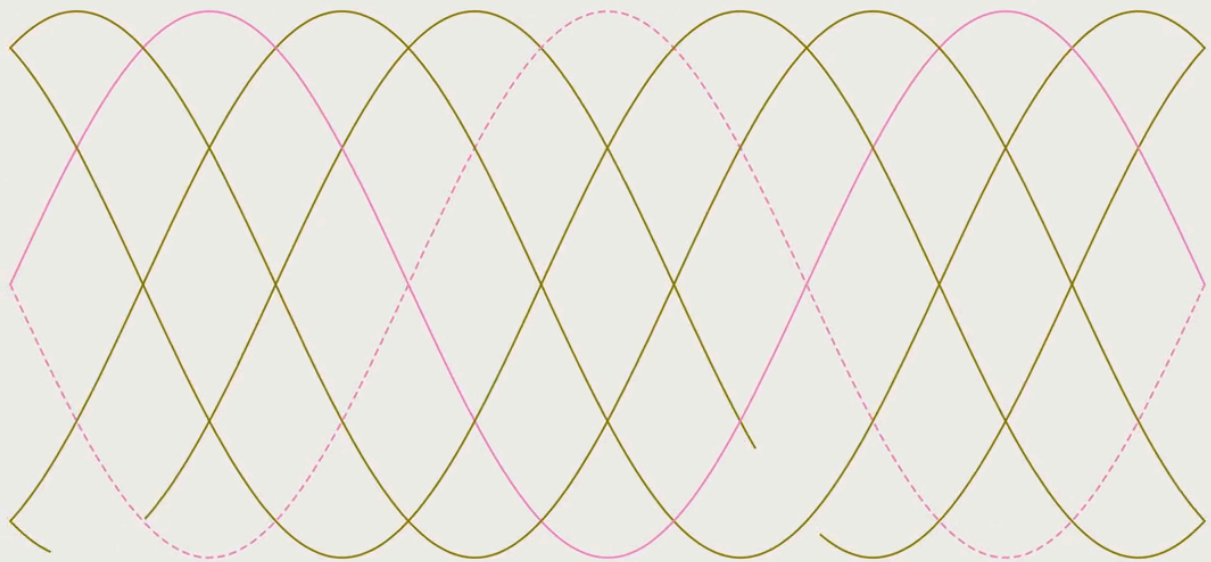
4 августа 2026 года · Исследование

Mixture-of-Kittens: наше мегаядро MoE с открытым исходным кодом для NVL72s

Мы открываем исходный код Mixture-of-Kittens — детерминированного обучающего мегаядра MoE для NVL72s, объединяющего коммуникацию и вычисления в одном ядре.



Стюарт Сал, Нэш Браун, Генри Вильдермут, Уильям Лин и Федерико Кассано · 28 минут чтения



Оглавление

Совмещение вычислений и передачи данных в MoE

Выбор правильного направления передачи данных

Правильная организация совмещения вычислений и передачи данных

Полная отмена синхронизации между центральным процессором и графическим процессором с помощью кольцевых буферов токенов

Другие функции и детали реализации



Сегодня мы открываем исходный код [Mixture-of-Kittens \(MoK\)](#), нашего производственного мегаядра для обучения с помощью метода Монте-Карло для NVL72.

По мере того как мы масштабировали обучение и логический вывод [Composer](#), нашей модели агентного кодирования, уровень «смесь экспертов» неизменно оставался основным узким местом. В зависимости от рабочей нагрузки и конфигурации обучения на него может приходиться более половины общего времени обучения.

MoK устраняет это узкое место, объединяя все коммуникационные и вычислительные процессы MoE в одно полностью детерминированное ядро. Теперь оно используется для обучения Composer на десятках тысяч графических процессоров.

Вы можете попробовать MoK и изучить код на [GitHub](#). Мы будем рады вашим отзывам и предложениям.

MoK появился в результате нескольких предыдущих попыток ускорить слой MoE. За последний год мы написали собственные ядра для обучения [MXFP8](#) и NVFP4 и разработали подход "[warp decode](#)" для логического вывода MoE.

Но эти методы оптимизировали только вычислительную часть уровня и предполагали, что обмен данными между графическими процессорами будет осуществляться отдельно. В наших производственных задачах обмен данными стал ограничивающим фактором. Это побудило нас полностью переработать уровень MoE, встроив обмен данными непосредственно в ядро.

Кроме того, переход на GB300 NVL72 изменил ситуацию в двух важных аспектах. Во-первых, NVL72 — это многоузловая стойка в рамках одного домена NVLink, обеспечивающая быстрое и точное наложение вычислений и обмена данными между всеми 72 графическими процессорами.

Во-вторых, встроенные процессоры Grace (буква «G» в GB300) работают медленнее по сравнению с графическими процессорами. Мы обнаружили, что потоки графического процессора легко догоняют работу центрального процессора, из-за чего графический процессор в это время полностью простаивает. Поэтому нам приходится максимально сокращать нагрузку на центральный процессор и синхронизацию между ними.

Нашим решением этих проблем стала Mixture-of-Kittens (MoK) — высокооптимизированное мегаядро для обучения с использованием метода Монте-Карло, созданное на основе базовых принципов для NVL72. MoK объединяет все коммуникационные и вычислительные процессы метода Монте-Карло в одном ядре, является полностью детерминированным и обеспечивает высочайшую производительность по сравнению с общедоступными реализациями.

NCCL + PyTorch DeepEP + PyTorch DeepEP + TransformerEngine
HybridEP + Megatron Mixture-of-Kittens

В нашем производственном обучающем стеке, состоящем из нескольких стоек NVL72, MoK увеличил количество токенов, обрабатываемых за секунду, в 1,41 раза.

В оставшейся части этого поста мы объясняем ключевые идеи MoK, в том числе рассказываем, как мы выбрали правильное направление коммуникации, структурировали взаимодействие между вычислениями и коммуникацией и устранили синхронизацию между центральным и графическим процессорами с помощью кольцевых буферов токенов. Мы также рассказываем о мегаядре, детерминированности, поддержке MXFP8 и некоторых других деталях реализации.

Пересекающиеся вычисления и коммуникация в MoE

MoK нацелен на слои MoE в стиле DeepSeek-V3 (DSV3), которые широко используются в моделях с открытым исходным кодом, включая GLM, Qwen, Kimi (до версии K2.7) и саму DSV. Эти слои объединяют одного общего эксперта с множеством специализированных экспертов, которых часто бывает несколько сотен.

Для каждого токена, поступающего на уровень MoE, маршрутизатор выбирает k лучших экспертов и присваивает каждому из них вес маршрутизатора. Затем каждый выбранный эксперт выполняет стандартные вычисления в сети прямого распространения, состоящие из восходящих и гейтовых проекций, активации SwiGLU и нисходящих проекций. Уровень объединяет выходные данные общих и маршрутизируемых экспертов с помощью весов маршрутизатора.

Мы используем следующие обозначения:

- E – набор маршрутизируемых экспертов (experts)
- Входной токен $x \in \mathbb{R}^D$
- Веса маршрутизатора $s \in \mathbb{R}^{|E|}$
- Веса эксперта $W_{up} \in \mathbb{R}^{I \times D}$, $W_{gate} \in \mathbb{R}^{I \times D}$, $W_{down} \in \mathbb{R}^{D \times I}$

И уровень MoE вычисляет:

$$\text{MoE}(x, s) = E_{\text{общий}}(x) + \sum_{j \in E} g_j E_j(x)$$

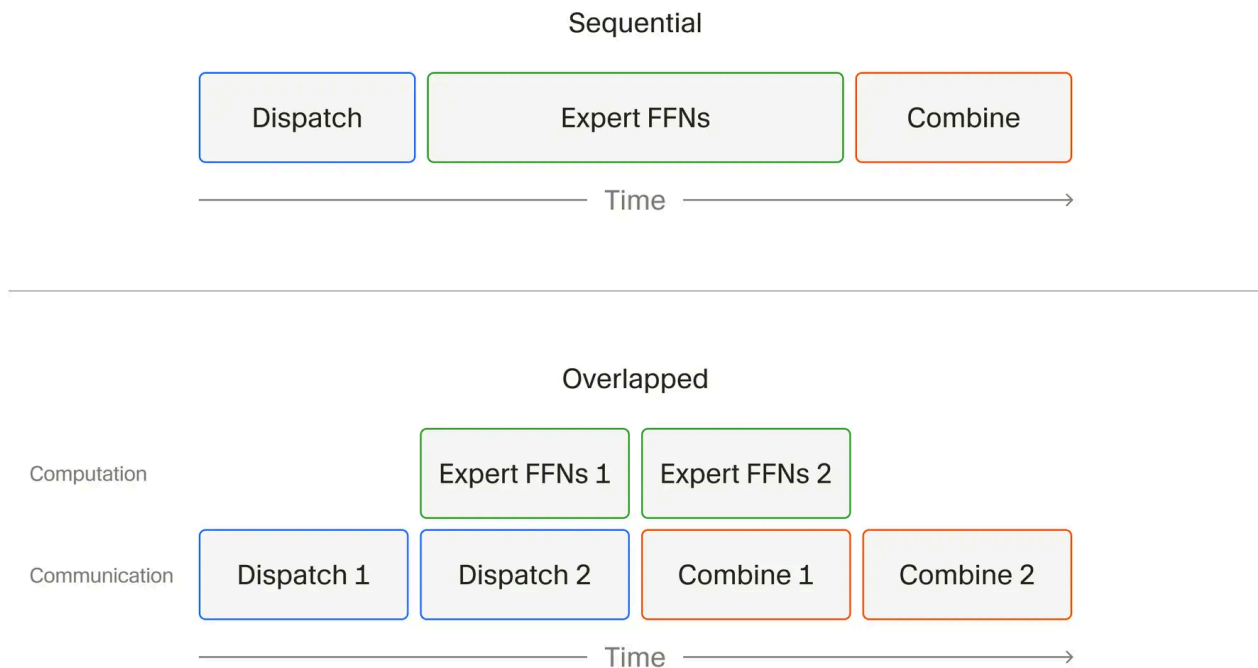
где

$$E_j(x) = W_{\text{вниз}}^{(j)} (\text{SiLU}(W_{\text{ворота}}^{(j)} x) \odot W_{\text{вверх}}^{(j)} x) \quad \text{и} \quad g_j = \frac{s_j}{\sum_{j \in E} s_j}$$

При экспертном параллелизме (expert parallelism, EP) мы разделяем маршрутизируемых экспертов и распределяем их веса между несколькими графическими процессорами, или рангами. Количество рангов, на которых в совокупности хранятся все веса экспертов, мы называем степенью EP. Например, при 256 маршрутизируемых экспертах и степени EP, равной 64, на каждом ранге хранятся 4 маршрутизируемых эксперта, а также общий эксперт. В результате токены должны передаваться между графическими процессорами до и после слоя MoE в соответствии с маршрутизацией.

При самой простой реализации распределенного метода экспертных оценок каждый токен отправляется в ранги, где находятся назначенные ему эксперты (отправка по принципу «все-всем»), выполняется функция FFN, результаты возвращаются в исходный ранг каждого токена (объединение по принципу «все-всем»), а затем вычисляется взвешенная сумма результатов экспертов. Но поскольку передача данных может занимать столько же времени, сколько и сами вычисления, последовательное выполнение этих двух операций неэффективно.

The standard remedy is to overlap dispatch/combine¹ communication with per-expert FFN through pipelining: transfer one chunk of tokens, compute FFN on it while overlapping transfer of the next chunk, and repeat. MoK is one variant of this scheme, with a set of novel, target-specific techniques that make it faster than existing baselines.



Choosing the right communication direction

When sending tokens across GPUs, one can choose a push-based mechanism, where the GPU that owns the tokens actively stores them into the remote destination GPUs, or a pull-based mechanism, where the GPU that needs the tokens loads them from the remote source GPUs. Existing approaches often rely on push-based communication for scattering and gathering tokens across GPUs (e.g., DeepEP).

The common notion is that pushing saturates inter-GPU links better, since it involves less protocol communication, and thus it becomes the default choice. Our observation, however, is that each mechanism has its own tradeoffs, and choosing the right one for each communication operator matters for maximizing performance, for the following three reasons.

Scheduling

To dispatch and combine tokens as fast as possible, the following conditions must hold:

1. All NVLink lanes interconnecting the 72 GPUs in the rack must stay saturated. We cannot afford stretches of time where tokens travel over only a subset of (source → destination) lanes, so tokens must be selected such that each source rank's sends are spread evenly across all destination ranks at any given moment.
2. Tokens sent to a rank should arrive ordered by that rank's local experts. If arrivals are unordered, the expert-grouped GEMMs wait longer for a full tile of tokens before the tensor core matrix multiplications can begin.

4. The overhead of satisfying the above three conditions must stay minimal. We want to spend most of the time actually sending tokens, and very little time scheduling or searching for tokens to send.

With push-based dispatch, we need to produce a schedule table with columns `{src_index, dst_rank, dst_index}`, where `src_index` is the index into the local incoming activation buffer and `dst_index` is the location in the destination rank's memory where the token must land. The row index of this table will decide the order in which tokens are sent over NVLink. We want the rows of this table to cycle through `dst_rank` round-robin, so that every connected lane stays busy.

Among the rows targeting a given `dst_rank`, the `dst_index` values must be interleaved evenly across all source GPUs, and they must also follow increasing local expert order on that destination rank. Building this table involves multiple sorts, and each rank's schedule must account for every other rank's, since no two source ranks can write to the same `dst_index`.

Schedule Idx	Src Idx	Dst Rank	Dst Idx
0	17	0	0
1	0	1	0
2	29	2	0
3	3	3	0
...	...	...	...

Push-based schedule: consecutive entries must interleave destination rank for full network saturation.

Schedule Idx	Src Rank	Src Idx
0	0	0
1	1	5
2	2	13
3	3	7
...	...	...

Pull-based schedule: consecutive entries must interleave source rank for full network saturation.

In theory, the number of rows in the table and in the destination buffer would match (which would require large memory allocation; more on this in a later section). No sorting is needed here. We walk over the router projection results, and whenever we find a token that should land on the current rank, we write its source rank and index into our schedule. The algorithm looks as follows:

Inputs: routing tensor $E \in \{0, \dots, RL - 1\}^{R \times N \times K}$, where $E_{r,i,k}$ is the global index of the expert assigned to the k -th route of token i on source rank r ; local rank c ; experts per rank L

Outputs: token counts per local expert $T \in \mathbb{N}^L$, token counts per local expert and source rank $M \in \mathbb{N}^{L \times R}$, region offsets $S \in \mathbb{N}^L$, schedule table Φ

```

 $T \leftarrow \mathbf{0}_L, M \leftarrow \mathbf{0}_{L \times R}$ 
for  $r = 0$  to  $R - 1, i = 0$  to  $N - 1, k = 0$  to  $K - 1$  do
     $e \leftarrow E_{r,i,k}$ 
    if  $\lfloor e/L \rfloor = c$  then
         $\ell \leftarrow e - cL$ 
         $T_\ell \leftarrow T_\ell + 1; M_{\ell,r} \leftarrow M_{\ell,r} + 1$ 
    end if
end for

```

```

 $S_0 \leftarrow 0$ 
for  $\ell = 1$  to  $L - 1$  do
     $S_\ell \leftarrow S_{\ell-1} + T_{\ell-1}$ 
end for

```

```

 $C \leftarrow \mathbf{0}_{L \times R}$ 
for  $r = 0$  to  $R - 1, i = 0$  to  $N - 1, k = 0$  to  $K - 1$  do
     $e \leftarrow E_{r,i,k}$ 
    if  $\lfloor e/L \rfloor = c$  then
         $\ell \leftarrow e - cL; o \leftarrow C_{\ell,r}$ 
         $p \leftarrow \sum_{r'=0}^{R-1} \min(M_{\ell,r'}, o) + |\{r' : r' < r \wedge M_{\ell,r'} > o\}|$ 
         $\Phi_{S_\ell+p} \leftarrow (r, iK + k)$ 
         $C_{\ell,r} \leftarrow C_{\ell,r} + 1$ 
    end if
end for

```

We can also reuse this schedule as-is for combine by choosing push-based combine, simply reading `{src_rank, src_index}` as `{dst_rank, dst_index}`. In fact, we can build the schedule once and reuse it for all four communication operations across forward and backward, by choosing:

- Pull-based forward dispatch
- Push-based forward combine
- Pull-based backward reverse-combine
- Push-based backward reverse-dispatch

The schedule is only a few megabytes in the worst case, so we can keep it around for reuse without any memory pressure. An additional benefit is that this completely eliminates inter-GPU, multi-lane signaling, as explained later below.

NVLink bandwidth utilization

As with any networking system, all user data (payload) being transferred over NVLink is sent with additional protocol metadata containing information about the source and destination, acknowledgements, etc., that depends on the networking protocol. While the details of the NVLink communication protocol are undisclosed, we can observe the data being sent on the link and reason about it carefully.

What we find is that push-based NVLink communication moves fewer total bytes (i.e., protocol metadata plus payload) and sends almost everything in one direction, so it achieves higher bandwidth utilization when all lanes are fully busy.

On the other hand, pull-based transfers move more bytes in total, but the protocol metadata is more split between both directions of the link. The puller first sends metadata one way, then receives more metadata plus the payload the other way.

We can verify this by writing a simple cross-GPU transfer kernel, and profiling it with NCU. For instance, in our microbenchmark sending one 256x256 BF16 tile (131,072B) over NVLink, we observe the following:

	Total	Total RX	Total TX	Protocol RX	Protocol TX	Payload RX	Payload TX
Push	159.6 KB	2.9 KB (1.84%)	155.6 KB (99.16%)	2.7 KB	24.6 KB	0 KB	131.1 KB
Pull	172.0 KB	147.5 KB (85.71%)	24.6 KB (14.29%)	16.4 KB	24.6 KB	131.1 KB	0 KB

saturation overall. However, NVLink has separate lanes for each direction. To reach the bandwidth advertised in the NVLink spec (1.8 TB/s for fifth-generation NVLink on GB300), traffic must fully flow both ways. And, for a workload like MoE dispatch/combine, each transfer tends to be only a few kilobytes and imbalanced, leaving execution bubbles on the link.

Empirically, pull-based communication delivers up to 29% higher NVLink bandwidth utilization than push-based communication when dispatching tokens under expert imbalance, making it the better choice.

Signalling

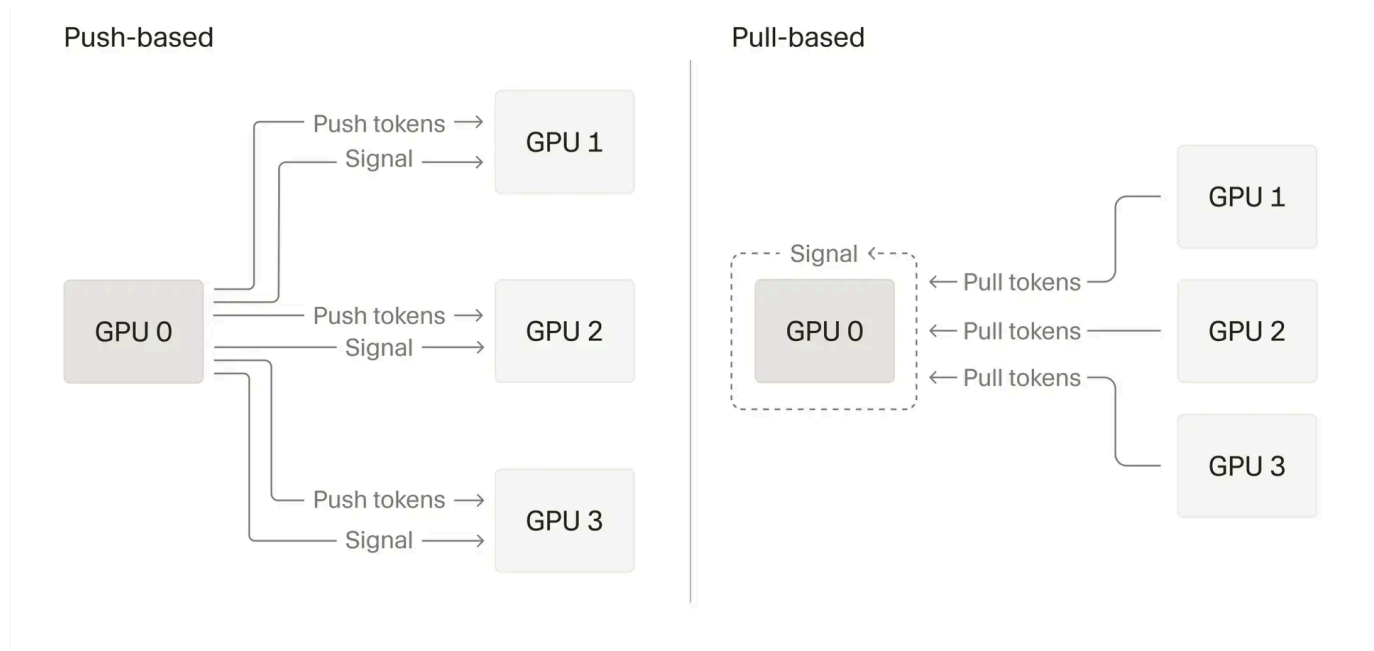
Signalling is necessary because the GPUs must know when dispatch/combine communication has completed. The GPUs can only begin operating on the received tokens after they receive a completion signal. It is important to minimize this signalling overhead, and choosing the right communication direction has a meaningful impact.

With push-based dispatch or pull-based combine, completion signals must cross between GPUs. Before an expert-grouped GEMM can begin, a rank must wait for signals from as many as 71 peers and flush memory across the rack by issuing a memory fence. In addition, the data is already sitting at the destination during the signal's transit time.

With pull-based dispatch and push-based combine, cross-GPU signalling disappears. Each rank issues a load, waits for the data to arrive, and can begin using it immediately. No multi-node synchronization is required. Also, only a single entity performs signalling, so the overhead does not grow with the degree of expert parallelism.

We found this signalling overhead to be quite large. In our multi-node microbenchmarks, push-based dispatch signalling incurs roughly 5.8x higher latency than pull-based dispatch signalling, at 103 μ s versus 18 μ s, and the cost accumulates quickly within a megakernel.

Based on these results, MoK uses pull-based forward dispatch, push-based forward combine, pull-based backward reverse-combine, and push-based backward reverse-dispatch.



Properly structuring the computation-communication overlap

Computation-communication granularity

Computation-communication granularity is the granularity at which computation and communication tasks signal each other of completion. It comes with an interesting trade-off.

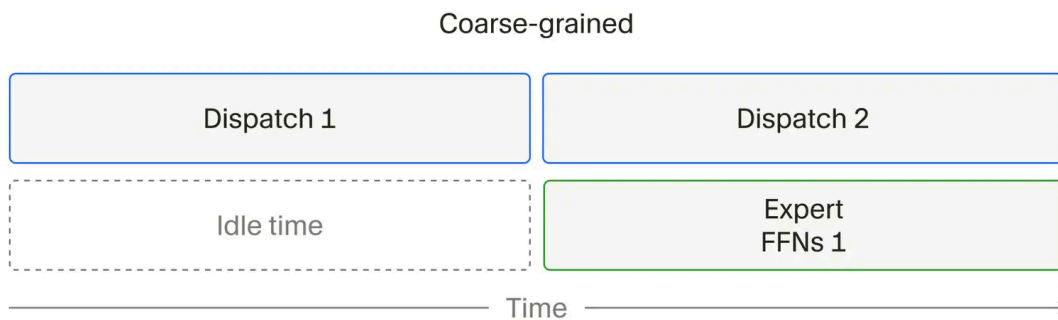
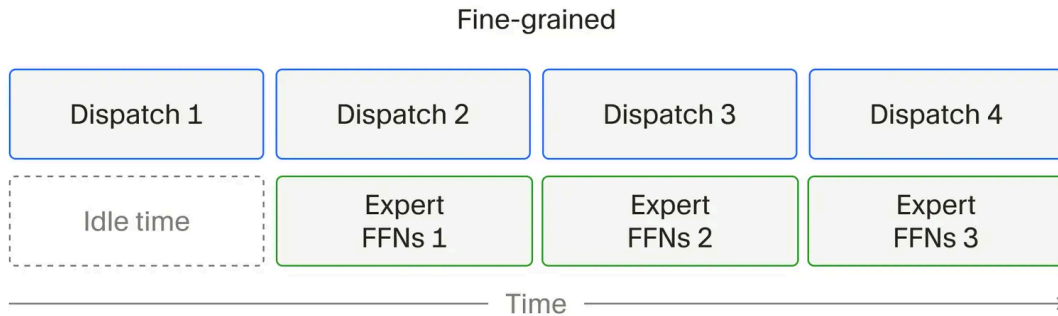
At one extreme, communication can be very fine-grained. We send only enough tokens for the tensor cores to launch a full matrix-multiply-accumulate instruction, 256 on Blackwell, for instance, and continuously overlap each transfer with the matrix multiplications. [Comet](#) is a good example of this approach.

At the other extreme, communication can be coarse-grained. We send thousands or tens of thousands of tokens at once and wait for the full batch to arrive before beginning the tensor core operations. [DeepEP](#) is a good example here.

Our observation is that neither extreme is the right choice. The optimal point sits somewhere in the middle and depends on the workload.

If the communication is too fine-grained, the tensor cores never fully saturate. Nvidia tensor cores are heavily pipelined, bandwidth-optimized accelerators, and we want to keep feeding them rather than hitting a barrier wait every few MMA instructions.

If the communication is too coarse-grained, the tensor cores wait too long for the first round of tokens to arrive and for the final round of outputs to be combined.



We call the set of tokens we send each round a **minibatch**. MoK makes the minibatch size a tunable parameter so that it's easy to sweep possible configurations before running a workload.

A rough heuristic for choosing the optimal minibatch size is to consider how many waves each operator (e.g., expert-grouped GEMM) requires.

A wave is one round of concurrent execution across all SMs on the GPU. If a work is too small (e.g., an expert-grouped GEMM receives too few tokens), we get a partial wave, in which some SMs sit idle. Alternatively, we may see a tail effect, where the final wave occupies only a fraction of the SMs.

We find it useful to choose a minibatch size that gives each expert grouped-GEMM operator in the FFN at least two full waves. This works for two reasons: (1) a single full wave is enough to fully saturate the tensor cores; and (2) with two waves, the second wave's MMAs overlap with the first wave's epilogue and dependent operators (e.g., the SwiGLU activation or the next grouped GEMM), amortizing the tail effect.

Specifically, on Blackwell GPUs, fully utilizing the tensor cores for an $M \times N \times K$ GEMM, where K is the reduction dimension, means each SM works on a 128×256 output tile, regardless of 1-SM or 2-SM MMA.

So given:

- T : number of tokens per transfer (i.e., the minibatch size)

2. Number of GMS

We want each GEMM to produce enough tasks to fill at least two waves, while keeping the communication granularity as small as possible for maximum overlap. For the up and gate projections, which run simultaneously since neither depends on the other, this means:

$$\frac{T}{128} \cdot \frac{2I}{256} \geq 2 \cdot C$$

Для проекции вниз, которая не может выполняться параллельно с другими операторами:

$$\frac{T}{128} \cdot \frac{H}{256} \geq 2 \cdot C$$

Объединяя эти два фактора, требование, которое мы ищем, является:

$$T \geq \frac{2C \cdot 128 \cdot 256}{\min(2I, H)}$$

Например, возьмем Kimi 2.5, базовую модель для Composer 2.5, где $H = 7168$ и $I = 2048$. На графических процессорах Blackwell $C = 148$, поэтому мы получаем:

$$T \geq 2368$$

В нашем микротестировании MoK для фигур Kimi 2.5 мы наблюдаем следующее, что примерно соответствует эвристическим алгоритмам:

T	ms
512	5.981
1024	4.669
1536	3.981
2048	3.666
2560	3.425
3072	3.447
3584	3.524
4096	3.473

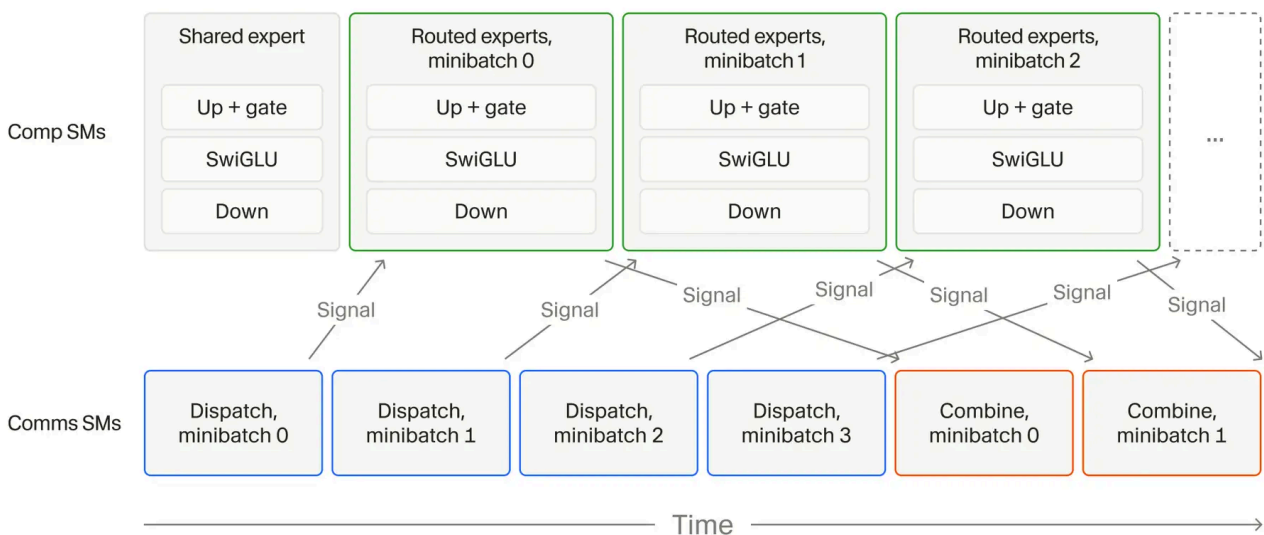
Scheduling the overlaps

employ the inter-SM overlapping technique, in which we assign some SMs to the expert FFNs (comp SMs) and some to dispatch/combine (comms SMs), and have the two groups signal each other through a local counter. This is possible because with Tensor Memory Accelerator (TMA) loads and stores, we can fully saturate NVLink bandwidth with less than a third of the SMs.

We use inter-SM overlapping as our base strategy. In the MoE forward pass, the comms SMs begin the pull-based dispatch all-to-all and signal the comp SMs each time a full minibatch of tokens has arrived and is ready for computation. The comp SMs then run MLP-SwiGLU atomically on those tokens, and once everything through the down projection is complete, they signal the comms SMs that the tokens are ready for combine.

Comms SMs go through all of dispatch tasks before doing any combine all-to-all tasks, so many completion signals arrive from the comp SMs by the time comms SMs are done with dispatch. Comms SMs will then move on to the push-based combine while the comp SMs finish the FFNs for the last-arriving tokens. Additionally, we can fit in shared expert FFN while the comp SMs wait for the first dispatch to complete.

Minibatch-only MoK forward schedule

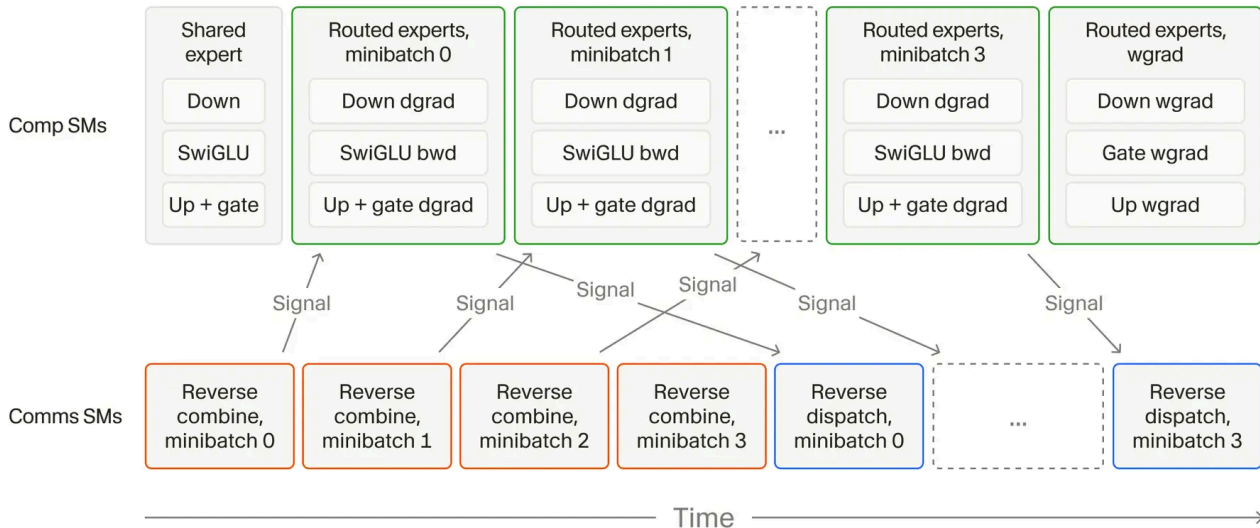


The backward pass is similar, though it gets a bit more complex with forward replay (more on this in the next section). The comms SMs perform the reverse-combine and signal the comp SMs to run the FFN backward, excluding the wgrads (i.e., 3 expert-grouped GEMMs for 3 dgrads and the SwiGLU activation backward), and the comp SMs signal the comms SMs once the FFNs are complete and the tokens are ready for the reverse-dispatch.

We perform the three remaining wgrads after the dgrads for all minibatches are complete. We delay the wgrads for two reasons: (1) reverse-dispatch does not depend on them, so wgrads can overlap

pass, we overlap the full shared expert backward with the first round of reverse-combine.

Minibatch-only MoK backward schedule



Fully eliminating CPU-GPU synchronization with ring token buffers

The last missing piece is handling MoE dynamism; we do not know in advance how many tokens will arrive at a destination rank.

There are two existing ways to handle this. The first is token dropping, where any tokens beyond a fixed buffer size are simply ignored and never enter the expert FFNs. The second is CPU-GPU synchronization, where the per-rank token counts computed from the router results are sent to the CPU, and the CPU uses them to allocate exact-sized buffers.

Both have obvious downsides. We do not want to drop tokens for the sake of training quality, and CPU-GPU synchronization prevents the GPU stream from running ahead of the CPU thread.

CPU-GPU synchronization is especially costly on GB300 NVL72s because the integrated Grace CPUs are very slow. Our traces often show GPU kernels bottlenecked by CPU work such as logging or pushing training metrics to a remote server, even though the same training stack caused no such problems on DGX machines with Intel CPUs. Given this, we want to minimize CPU-GPU synchronization as much as possible.

In MoK, we resolve this with a ring token buffer, which we call **macrobatch**. Instead of allocating a large, mostly unused destination buffer for the comms SMs to transfer tokens into, we create a fixed-size ring buffer of a few hundred megabytes and cycle through it at minibatch granularity.

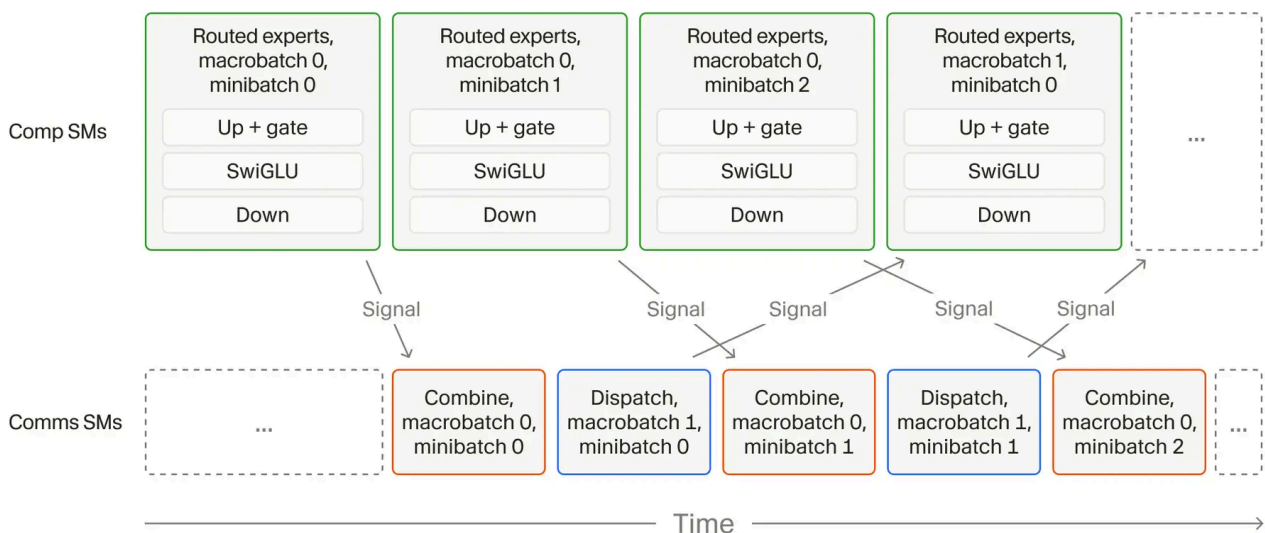
yet arrived. Ideally, we overlap across the ring boundary, computing on the tail end of the buffer while dispatching into its start.

The key idea is to drain each slot as early as performance allows so that it can be reused as early as possible. In the forward pass, dispatch fills slots and combine drains them, freeing them for the next dispatch.

However, if we run all dispatches followed by all combines (as in the previous section's schedule), the next macrobatch cannot begin until every combine has finished, and computation and communication only start overlapping again after its first dispatch completes.

To avoid this, we perform dispatch-combine interleaving for the forward pass. The combine for a minibatch of the previous macrobatch is interleaved with the dispatch for the corresponding minibatch of the next macrobatch, which uses the same region of the ring buffer. This way, the buffer is refilled as soon as it is emptied. Note that the first macrobatch's dispatches and the last macrobatch's combines are not interleaved, as they do not have their counterparts.

MoK forward schedule at a macrobatch boundary



Reversed ring to minimize forward replay during backward

The backward pass is slightly more complex because the backward megakernel must support forward activation replay. The macrobatch buffer is a ring, so if there are multiple macrobatches, some saved activations are overwritten during the forward pass and must be recomputed. The replay only needs to run up to the SwiGLU activation, since backward only needs the down projection's input activation and output gradient, not its forward output.

with more activations to replay. MoK therefore walks through the tokens in reverse macrobatch order during the forward pass, so that the saved ring buffer is always either completely full or contains all of the tokens, minimizing forward replay during the backward pass.

Other features and implementation details

Megakernel

We built MoK using the [megakernel](#) technique, where instructions overlap at the SM task level instead of being separated by kernel launch boundaries. Megakernels are usually suited for fusing an entire forward or backward step, but we found the technique useful for MoK for two reasons: (1) implementing minibatching and macrobatching without megakernel fusion would require multiple kernel launches, increasing the launch boundary overhead; and (2) for inter-SM overlapping, we found multiple streams with green contexts unreliable at partitioning SMs exactly as intended, while software partitioning gives exact allocation guarantees.

Determinism

For internal ablations and on-policy RL post-training, we designed MoK to be fully deterministic. The order of floating point operations is fixed, so the same input produces bitwise-identical output regardless of hardware scheduling and instruction issue order.

Cluster Launch Control (CLC) for RDMA overlap

The MoK megakernel schedules its tasks through CLC to allow RDMA overlap. CLC is a hardware-native work-stealing feature introduced in Blackwell as a new way of implementing persistent grid kernels. A CLC-based kernel can naturally yield to kernels on a higher-priority stream without having to wait for the current kernel to fully complete. This matters because, during training, we often need to overlap intra-rack communication and computation with inter-rack InfiniBand/RoCE communication, such as FSDP all-gather, which require SMs of their own. Without CLC, the inter-rack communication would serialize behind the megakernel, causing significant performance degradation.

MXFP8 support

MoK supports both BF16 and MXFP8 precision modes. We train in MXFP8 mode, since it is faster and causes no numerical issues. In MXFP8 mode, the shared expert stays in BF16, as we found this to affect the training stability.

One added cost of MXFP8 is that tensors must be quantized before they are fed into the tensor cores. To minimize this overhead, we (1) provide an optimized MXFP8 quantization kernel for pre-quantizing the weights; and (2) fuse activation quantization into the dispatch all-to-all, the expert-grouped GEMMs, and the SwiGLU.

calculation. Instead of saving the full down-projection output, it computes them from the inner product of the SwiGLU activation and the down-projection dgrad. This computation is fused into the SwiGLU backward to avoid additional memory traffic.

Computation/Communication SM tuning

The optimal ratio of comp to comms SMs depends on the workload. The local token count, the model shapes, and the hardware compute and network bandwidth all change how long the dispatch/combine tasks take relative to the expert FFN tasks. Even within the same workload, the optimal ratio differs between the forward and backward passes. MoK therefore exposes tunable parameters that set the number of comms SMs separately for forward and backward.

Speedups

We evaluated MoK with two types of benchmarks: individual MoE layer benchmarks, for which we release all benchmark code, and end-to-end benchmarks on our internal production training stack across multiple NVL72 racks.

MoE layer benchmarks

We benchmark single MoE layer execution: global token scheduling, token dispatch all-to-all, expert FFNs, token combine all-to-all, and the final weighted sum. We benchmarked the forward and backward passes separately, in both BF16 and MXFP8 modes. All tests ran within a single NVL72 rack, with EP degree 64 and 2,048 tokens/GPU before routing.

We chose baselines that (1) support both the MoE forward and backward passes; and (2) run on NVL72s:

- NCCL + PyTorch
- DeepEP + PyTorch
- DeepEP + TransformerEngine
- HybridEP + Megatron, which is Nvidia's recommended option on NVL72s over DeepEP + Megatron

We tested the shapes of widely used open-weight models:

- Kimi K2.7 Code shape (E : 384, H : 7168, I : 2048, top-k: 8)
- GLM-5.2 shape (E : 256, H : 6144, I : 2048, top-k: 8)
- Qwen3.5-397B-A17B shape (E : 512, H : 4096, I : 1024, top-k: 10)
- DeepSeek-V4-Pro shape (E : 384, H : 7168, I : 3072, top-k: 6)

The graphs below show results for all four shapes across all baselines:

NCCL + PyTorch DeepEP + PyTorch DeepEP + TransformerEngine
HybridEP + Megatron Mixture-of-Kittens

Expert-parallel MoE throughput: MXFP8 backward (GB300 NVL72s)

NCCL + PyTorch DeepEP + PyTorch DeepEP + TransformerEngine
HybridEP + Megatron Mixture-of-Kittens

Expert-parallel MoE throughput: BF16 forward (GB300 NVL72s)

NCCL + PyTorch DeepEP + PyTorch DeepEP + TransformerEngine
HybridEP + Megatron Mixture-of-Kittens

Expert-parallel MoE throughput: BF16 backward (GB300 NVL72s)

NCCL + PyTorch DeepEP + PyTorch DeepEP + TransformerEngine
HybridEP + Megatron Mixture-of-Kittens

Overall, we observe that MoK is up to 2.37x faster for the MXFP8 forward, 1.78x for the MXFP8 backward, 1.92x for the BF16 forward, and 1.58x for the BF16 backward, compared to the fastest baseline.

End-to-end benchmarks

In addition to the benchmarks against publicly available implementations, we report speedups on our internal production training stack. Previously, our training stack relied on a DeepEP-based implementation with custom MXFP8 MoE computation kernels for expert-parallelism. We compare the end-to-end tokens per second metric by varying the MoE layer implementation choice (DeepEP vs MoK). The benchmarks were conducted on 512 GPUs across several GB300 NVL72 racks.

End-to-end training throughput comparison (GB300 NVL72s)

DeepEP-based Mixture-of-Kittens

	DeepEP-based	MoK
Tokens / second / GPU	760.9	1,070.2 (1.41x)

Overall, MoK delivered an approximately 41% tokens-per-second speedup over our previous DeepEP-based production setup, allowing us to train our models more efficiently across our GB300 NVL72 infrastructure.

Megakernels for the agent era

Mixture-of-Kittens is fully open-sourced as of today. We plan to keep maintaining it and welcome community feedback and contributions. Our hope is that this lowers the barrier to AI research, allowing more researchers and labs to train models efficiently. MoK also aims to be flexible. We put a lot of effort into making it easy to modify, so that with the help of agents, you can quickly adapt it to platforms beyond the ones we run on.

As a side note, with the arrival of AI models capable of writing performant kernels, the kernel optimization space has become a lot more interesting. Single-operator kernels that used to be written by hand are now almost entirely handled by AI. Given the right design directions (e.g., how to specialize warps, which PTX instructions to use), agents can one-shot state-of-the-art kernels.

Even more interesting, however, is the new capabilities this gives us. Last year, writing a megakernel required creating a layer of simplified abstractions like this one. This year, we did not need a framework. Instead, we removed the additional layer of abstraction and, with the help of agents, worked through the complexity ourselves from scratch.

Agents automated the simpler tasks (e.g., single-operator kernels) and helped us get through the harder ones (e.g., writing a distributed MoE megakernel) much faster and with a very small team.

would love to hear from you. Reach out to us at timmy@cursor.com.

Citation

If you use this work, please cite:

Stuart H. Sul, Nash Brown, Henry Wildermuth, William Lin, and Federico Cassano. "Mixture-of-Kittens MoE Megakernel for NVL72s." Cursor Research, Aug 2026.
<https://github.com/cursor/mixture-of-kittens>

Jul 14, 2026 · Research

Grok 4.5 Model Card

 Cursor Team · 1 min read

`EMBO {OUTEREMON,`

Jul 8, 2026 · Research

Introducing Grok 4.5

 Cursor Team · 3 min read

`and William Lin and F`

Aug 29, 2025 · Research

1.5x faster MoE training with custom MXFP8 kernels

 Stuart Sul · 22 min read

`-kittens}}},`

Wang for reading this

[View more posts →](#)

1. In this post, dispatch refers to the dispatch all-to-all communication and combine refers to the combine all-to-all communication. The weighted reduction usually included in "combine" is referred to separately as the weighted sum. ↩

Product	Resources	Company	Legal	Connect
Agents	Download	Careers	Terms of Service	X
Teams	Changelog	Blog	Privacy Policy	LinkedIn
Enterprise	Docs	Community	Data Use	YouTube
Pricing	Learn	Students	Security	
Code Review	Value Calculator	Brand		
CLI	Forum	Future		

